

CSE/DSC 234 Project 2 – Schema Linking with SFT

Final Report

Final validation leaderboard: 0.7107 (Table 0.7791 / Column 0.6423). Baseline (smoke adapter): 0.5605. Best single adapter: 0.6648 (sweep 13). Submission: `python3 main.py --input X --output Y` runs a 3-way LoRA ensemble *across two base models AND mixed table retrievers per adapter*: sweep 13 on Qwen-Coder-1.5B with embedding retrieval, sweep 15 on Qwen-Coder-1.5B with BM25 retrieval, sweep 22 on Qwen3-1.7B with embedding retrieval. Completes 101 questions in **12m14s** on a 24 GB MIG slice (<15 min budget). `main.py` reads each adapter’s `adapter_config.json` to group adapters by base model (so each base is loaded only once) and accepts a `path[:retriever]` spec per adapter so the same adapter under different retrievers can be ensembled too.

1 Training Data Methodology

The released 301 train + 101 validation split was the starting point. We treated schema linking as next-token prediction over a chat prompt: a fixed system message explains the JSON contract and wildcard convention; the user turn carries the serialized schema and NL question; the assistant turn carries the canonical JSON target (sorted keys + sorted columns) emitted by `prompt.target_string`.

Schema serialization. The baseline serializer is one line per table, `TableName: c1, c2, ...`. We tested two richer variants: (i) **types** – appending column type in parentheses; (ii) **keys** – * for primary keys and `->TargetTable.col` for foreign keys. A combined **types_keys** serialization roughly doubled token counts on the dense SBO schemas; on the largest (SBO-Banking) the prompt exceeded 6k tokens. **keys** alone added no tokens on SBO because the SBO Spider schemas carry no PK/FK metadata – only 5 of the 17 schemas do (ATBI, NTSB, NorthernPlainsFireManagement, KlamathInvasiveSpecies, PacificIslandLandbirds).

Output format. Raw JSON with stable casing, alphabetical keys and column lists. A table with no specific columns (wildcard queries like `count(*) from t`) is emitted as `{"t": []}`, not omitted. We verified this end-to-end with `tests/test_checklist.py`; only 3 validation questions have empty-gold tables but failing to emit them is a recall miss on both axes.

Table-level retrieval at training time. Schemas with > 500 columns (all 9 SBO modules + NTSB + NYSED) get BM25-filtered to the top-20 tables. **Oracle augmentation:** at training time we force-include every gold-referenced table in the kept set so the model is never asked to predict labels absent from its prompt. Inference does not have this oracle.

Augmentation. The 9 SBO modules ship with only 6–12 train examples each (~22% of train) versus 24–60 for the parks/NTSB domains; SBO is also where ~55% of our validation table-misses concentrate. We generated 216 additional (question, gold_sql) pairs via the Claude API (Haiku 4.5; 9 batches of 24) focused entirely on SBO, labeled each with the project’s `sql_to_schema_links.py` (sqlglot-based) extractor, and dropped any that failed to parse or referenced 0 schema tables. Only 1 of 217 candidates was dropped (a duplicate question; 0 hallucinated tables, 0 parse errors – evidence the prompt and validation pipeline are tight). The merged `train_augmented.json` has 517 examples. The audit log lives at `data/train_augmented_audit.json`.

2 Final Pipeline Architecture

Base model. Qwen/Qwen2.5-Coder-1.5B-Instruct. The code-pretrained variant of Qwen2.5-1.5B; same parameter count, same ChatML template, same tokenizer; SQL/schema pretraining produced measurable gains over the vanilla Instruct variant (Sec. 4).

Adapter. LoRA $r=32$, $\alpha=64$, dropout 0.05, target modules `{q,k,v,o,gate,up,down}_proj` (the full attention block plus the MLP block). bf16 with gradient checkpointing during training. Adapter safetensors are shipped as **fp16** (PEFT’s default fp32 save was 148 MB per adapter, above GitHub’s 100 MB per-file limit; fp16 halves that to 74 MB and we verified the score is preserved within ± 0.001 of fp32). The 3 adapters together fit in ~220 MB of repo size.

Optimizer. AdamW (TRL default), `lr_scheduler_type=linear`, `warmup_ratio=0.05`, `per_device_train_batch_size=1`, `gradient_accumulation_steps=4`, `max_steps=200`, `max_length=4096`. Three adapters were trained at $\text{lr} \in \{3e-4, 3e-4+aug + 40e-4\}$ – their union forms the ensemble.

Inference pipeline (main.py). (1) Read each adapter’s `adapter_config.json` to determine which base model

it requires; group adapters by base model. (2) For each group, load that base model once and load every adapter in the group as a NAMED PEFT adapter via `PeftModel.load_adapter`. (3) For each adapter in the group: `model.set_adapter(name)`, run all 101 queries through that adapter, accumulate {table→cols} predictions. (4) Free the group’s base before loading the next group. (5) Union the accumulator across all adapters and write final predictions. **Partial-save after each adapter:** a >15min timeout still yields a smaller ensemble of completed adapters. **Retrieval:** embedding-based – BAAI/bge-small-en-v1.5 sentence-transformer cosine similarity between the question and each table’s “table tokens columns column-tokens” document; top-20 tables kept.

Post-processing. `canonicalize_prediction` drops any non-schema identifier (precision-preserving hallucination filter) and re-cases identifiers to schema casing. The final submitted output had 0 schema-invalid identifiers across all 101 validation predictions.

3 Experimentation Methodology

We ran 20 numbered sweeps via **RapidFire AI** (`train_rapidfire.py`, `RFGGridSearch/RFSFTConfig`). Sweeps 1–9 were spent surviving infrastructure problems (described below); sweeps 10–20 produced the substantive results in Table 1. Per-sweep `rapidfire.log`, `training.log`, and extracted MLflow metrics (`metrics.json`) live under `logs/sweepN/`; full per-run configs are pulled from the rapidfireai MLflow store on demand.

3.1 Sweep dimensions

Across the three *training-time* axes – base model, adapter capacity, training data – and four *inference-time* axes – prompt style, retriever, ensemble strategy, post-processing – we ran 11 distinct training configs and ~10 distinct inference-time variants. Configs were chosen one-at-a-time A/B style off the current champion so each result is attributable to a single knob. Multi-config sweeps (8–16 in `RFList`) were used in our first attempts (sweeps 5–8) but, after infrastructure issues forced us to `num_chunks=1` (Sec. 3.2), we switched to single-config sweeps and let the cost of more sweeps buy clean attribution.

3.2 Infrastructure issues and how we worked around them

Issue #1: DSMLP cleanup SIGKILLs (~24h of lost GPU time). Sweeps 5–8 ran multi-config `RFGGridSearch` with 8–16 configs and `max_steps=300`. Each tried to train through ~5–6h of wall time. DSMLP enforces a process-cleanup window we did not anticipate – somewhere around the 4–6h mark, our worker process got SIGKILL’d with no warning. The mlflow metrics survived (they are written to a SQLite DB on `$HOME`) so we could see that sweep 8 had reached step 264/300 across all 8 configs before dying. But the trained LoRA weights themselves had *nowhere on disk*. We initially assumed the problem was the kill itself; the real story turned out to be deeper.

Issue #2: RapidFire’s SHM-only checkpoint store. While debugging the loss, we ran sweep 9 with only 2 configs and `max_steps=200`, expecting it to finish in well under the kill window. It did – “Experiment ended” fired cleanly, no SIGKILL. *We still had no final_checkpoint/*.safetensors on disk*. Reading `rapidfireai/fit/backend/worker.py` around L440 revealed why. By default RapidFire writes intermediate checkpoints to `/dev/shm` via its `shm_manager`; the disk write only fires inside a single branch gated on `is_run_finished == (chunk_id == num_chunks-1) && (steps ≥ total_steps)`. With the controller’s round-robin chunk schedule, a run typically hits its step budget on chunk N where $N < \text{num_chunks}-1$, the disk-save branch is skipped, and the in-SHM adapter dies with the worker process at `experiment.end()`. We considered patching the constant `USE_SHARED_MEMORY = True` at module level but rejected that as fragile (it affects all rapidfireai usage on the system and would not survive a `pip install --upgrade`). The clean workaround: pass `--num_chunks 1` so every run necessarily ends on “the last chunk” by construction and the disk-save branch fires reliably. All sweeps from 10 onward use this workaround; sweep 10 was the first successful end-to-end persisted RapidFire training in our series.

Issue #3: HuggingFace Trainer fallback (`train_single.py`). While debugging Issue #2, we were not sure whether the problem was orchestration (rapidfireai) or training (TRL/PEFT setup). We built `train_single.py` – a ~120-line plain `trl.SFTTrainer` script – as an A/B against the same config. It trained fine in 14 minutes and produced a 0.6078 adapter, confirming that the training stack worked end-to-end and isolating the failure to the rapidfireai chunked-orchestration path. After we landed Issue #2’s fix this script became dead code, but it stayed in the repo as a stress-tested single-config fallback.

Issue #4: CUDA SIGFPE in `main.py` at `batch_size>1`. On the first ~0.61 inference run we hit a hard “Floating point exception (core dumped)” right at the first `model.generate` call. Process crashed before producing any predictions. We narrowed the trigger by bisecting the CLI flags and found that batch size 1 worked, ≥ 2 SIGFPE’d. Stack combination is `transformers 4.57 + torch 2.10 + bf16 + left-padded prompts ~4k tokens + H100`.

We did not find a root-cause issue but the pattern matches reports of a kernel edge case in the attention path with long left-padded sequences in bf16. We defaulted `batch_size=1` in `main.py` (with a comment) and noted it in the README so a grader doesn't trip over it.

Issue #5: Mismatched val-data layout. Halfway through development we reorganized the repo to push raw data under `data/`; the `tests/test_checklist.py` script (the rubric quick-start verifier) was written against the old root-level paths. We updated the test, kept it green, and added it to the development loop – it's a cheap sanity check that every commit doesn't break the I/O contract (per-file `question_ids`, schema-folder layout, wildcard-table preservation, hallucination filtering).

3.3 The supporting tools we built

- `augment_data.py` – the SBO augmentation pipeline; calls Claude Haiku 4.5 in 9 batches, validates each generated SQL by running it through the project's `sql_to_schema_links.py` extractor, writes both the merged `train_augmented.json` and a per-candidate audit log.
- `format_training_data.py` – converts raw JSON splits to the JSONL `messages` format consumed by TRL/RapidFire, parameterized by `--prompt_style`.
- `format_two_stage_data.py` – splits the joint task into stage-A (table prediction) and stage-B (per-table column prediction) datasets for the two-stage experiment.
- `expand_columns.py` – the keyword-based column-expansion post-processor (Sec. 4 below).
- `generate_cot_traces.py` – calls Claude Haiku 4.5 once per train example to produce a brief, schema-grounded rationale; validates that the rationale references at least one gold table; produces `data/train_cot.json` for sweep 21's CoT-distillation experiment.
- `reproduce_champion.py` – self-contained script that re-derives the 3-way ensemble predictions from the per-model `preds_sweep{13,15,18}_embed.json` files; can also regenerate those from the adapter weights via `--regenerate`. Saves us from having to re-run inference just to regenerate the champion submission.
- `logs/README.md` – per-sweep mapping from sweep number to config knobs, status, best train metric. Built from the MLflow store via a small script that pulls `runs` $\bowtie$ `params` $\bowtie$ `metrics` for each experiment.

3.4 Inference-time experimentation

Once we had the single-best adapter (sweep 13), we tested seven inference-time variants: `max_tables` 20→40, three retrievers (BM25 / embed / hybrid via RRF), self-consistency at `K=3` (union-of-samples and majority-vote), column-expansion post-processing, and union/intersection/ majority-k ensembling over progressively more adapters (2–6 way). Variants were A/B'd directly against the previous champion file in `predictions/` – no retraining, ~5–15 min per A/B. Many were negative (Sec. 4) but the embedding retriever (+0.014 single, +0.016 in the ensemble) was the single most valuable inference-time discovery.

4 Results and Analysis

What worked.

- **Switching to Coder-1.5B (+0.003):** small but in the right direction; sets up the bigger gains.
- **Bigger LoRA: `r=32` + MLP modules (+0.039):** the single biggest training-time lever. The full attention+MLP adapter has $\sim 3.5\times$ more trainable parameters than `r=16` q/k/v/o and converged to lower train loss (0.075 vs 0.108) without overfitting at 200 steps. Notably this is the configuration our sweeps 5–7 attempted but lost to the SHM bug – the `num_chunks=1` workaround was load-bearing for actually *getting* this number.
- **Embedding-based table retrieval (+0.014 single, +0.016 ensemble):** BAAI/bge-small-en-v1.5 surfaces semantically related tables that BM25 misses. Diagnostic example – qid 95 needs ORCT (the incoming-payments header table); BM25 ranks ORCT outside top-10 (all top hits are PWZ* payment-wizard children); the embedder puts ORCT at rank 1 (cosine 0.728). This was the biggest *inference-time* lever.
- **Union ensemble of 3 diverse adapters (+0.034 over best single):** sweep 13 (`lr=3e-4`, 301 train), sweep 15 (`lr=3e-4`, 517 train, 400 steps), and sweep 18 (`lr=2e-4`, 301 train). Chosen because their errors are sufficiently uncorrelated; 4- and 5-way variants regressed because the weaker fourth model (sweep 20 / sweep 12) brings more noise than signal.
- **Adding Qwen3-1.7B (sweep 22) for cross-family ensemble (+0.017 over single-base champion):** sweep 22 alone scores 0.66 – weaker than sweep 13 (0.66) – but its errors are markedly less correlated with the Qwen-Coder ensemble members because it comes from a different model family entirely. Replacing sweep 18 (the weakest Coder member) with sweep 22 produced the 3-way 13+15+22 all-embed ensemble at 0.7019. The submission's `main.py`

#	base model	key knobs	lr	steps	val LB	Δ
10	Qwen2.5-1.5B	r=16 q/k/v/o, BM25	3e-4	200	0.6229	—
12	Qwen2.5-Coder-1.5B	r=16 q/k/v/o, BM25	3e-4	200	0.6258	+0.003
13	Qwen2.5-Coder-1.5B	r=32 + MLP, BM25	3e-4	200	0.6648	+0.039
18	Qwen2.5-Coder-1.5B	r=32 + MLP, BM25	2e-4	200	0.6609	−0.004
19	Qwen2.5-Coder-1.5B	r=32 + MLP, BM25, cosine LR	3e-4	200	0.6479	−0.017
14	Qwen2.5-Coder-1.5B	r=32 + MLP, BM25, 517 train (aug)	3e-4	200	0.6379	−0.027
15	Qwen2.5-Coder-1.5B	r=32 + MLP, BM25, 517 train, 400 steps	3e-4	400	0.6600	−0.005
16	Qwen2.5-Coder-1.5B	r=32 + MLP, BM25, 301 train, 400 steps	3e-4	400	0.6268	−0.038
11	Qwen2.5-1.5B	r=16 q/k/v/o, types_keys prompt	3e-4	200	0.5946	−0.028
20	SmolLM2-1.7B	r=32 + MLP, BM25	3e-4	200	0.5301	−0.135
17a/b	Qwen2.5-Coder-1.5B	Two-stage: table model + column model	3e-4	200	0.6127	−0.052
21	Qwen2.5-Coder-1.5B	r=32 + MLP, CoT-distilled training data	3e-4	200	0.6103	−0.054
22	Qwen3-1.7B	r=32 + MLP, BM25, original data	3e-4	200	0.6599	−0.049
Inference-time – best single adapter (sweep 13):						
BM25 retrieval (baseline)						0.6648
max_tables: 20 $\rightarrow$ 40 (more distractors)						0.6032
Embedding retrieval (BAAI/bge-small-en-v1.5)						0.6786
Hybrid retrieval (RRF of BM25 + embed)						0.6724
Self-consistency, K=3, T=0.5, union-of-samples						0.6737
Column expansion (question-keyword $\leftrightarrow$ col-name match)						~same
Ensembles – 3-way Coder-only (sweeps 13+15+18):						
BM25 retrieval						0.6986
Embedding retrieval (single-base champion before Qwen3)						0.7002
Hybrid retrieval (RRF)						0.6977
2-way 13+18, 4-way +14 / +20 / +11						0.66–0.69
Ensembles – cross-family (Coder + Qwen3):						
2-way 13+22 (both embed)						0.7063
3-way 13+15+22 all-embed (single-retriever)						0.7019
3-way mixed-retriever: 13_embed + 15_BM25 + 22_embed						0.7107
(submitted champion; 0.7147 in the offline probe – the 0.7107 is the actual fp16 end-to-end)						

Table 1: All 11 distinct training configs (top) and the inference-time ablations on the best single adapter and the 3-way ensemble (bottom). Bold marks the single best config in each section. Δ is vs sweep 10 (the single-config baseline). **r=32 + MLP** means LoRA r=32 with target modules {q,k,v,o,gate,up,down}_proj. **BM25** retrieval scores tables by query-vs-(table+cols) BM25; **embed** replaces BM25 with sentence-transformer cosine similarity (BAAI/bge-small-en-v1.5); **hybrid** is Reciprocal Rank Fusion of the two.

groups adapters by base model and loads each base only once, so this multi-base ensemble still fits in the 15-minute inference budget.

- **Mixed-retriever ensemble (+0.009 over the all-embed champion):** the same adapter produces different predictions under different retrievers (BM25 vs embed vs hybrid) because the candidate-table set in the prompt differs. Probing this axis, we found that putting sweep 15 under BM25 retrieval (while sweep 13 and sweep 22 stay on embed) added a measurable boost: the 3-way mixed-retriever ensemble **13_embed + 15_BM25 + 22_embed** lands at **0.7107** – our final submitted leaderboard. The intuition: “which retriever is right for this adapter” is not just about training-time consistency; for ensemble purposes, the right pairing is whichever makes the errors of this member most uncorrelated with the others. We expose this in `main.py`’s `--ensemble_dirs` via a `path[:retriever]` spec per adapter.

What didn’t work, and why.

- **types_keys prompt (-0.028):** roughly doubled token count and *regressed* both table and column scores. We attribute this to attention dilution – with $\sim 2\times$ more schema text per prompt, the 1.5B model focuses less precisely on the 2–4 actually-relevant tables out of 20. Cheap signal at this scale beats expensive signal.
- **max_tables=40 (-0.020):** contrary to our prior that exposing more candidate tables would recover BM25-cut gold tables. Distractor tables hurt the model’s table precision more than the marginal recall recovery.
- **Two-stage (table model then column model) (-0.052):** a clean architectural negative result. Stage-A loses the column signal that disambiguates similarly-named SBO siblings (OPMG vs OPHA); stage-B cannot reason cross-table

about JOIN columns; errors compound (a wrong table in stage-A can't be corrected by stage-B). Joint reasoning is load-bearing for this task.

- **SmolLM2-1.7B base swap (sweep 20, -0.135 alone)**: the model is genuinely weaker on this distribution (token accuracy 0.80 vs Coder's 0.90); adding it to the ensemble *hurt* (-0.037), so diversity does not always help when the noise floor of the new member is too high.
- **Data augmentation alone (sweep 14, -0.027)**: train_loss roughly 2× higher than the 301-example baseline at the same 200 steps – underfitting on the bigger dataset. Doubling steps (sweep 15) recovered most of the regression but did not exceed the 301-example baseline. The matched-step ablation on the original 301 data (sweep 16, lr=3e-4, 400 steps) *also* hurt (-0.038) – the model overfits on 301 examples at 400 steps. The 2×2 table-size/step-count grid implies the augmented data distribution is roughly neutral for this base+adapter; the limiting resource is capacity matched to per-example exposure, not raw data quantity.
- **Self-consistency union-of-samples (-0.005)**: at T=0.5 sampling diversity adds more noise than signal; the model's greedy decode is already quite consistent.
- **Column-expansion post-processing (neutral)**: “≥1 token overlap” was too aggressive (-0.009 from added false positives); the stricter “≥2 meaningful tokens” rule only fires on ~9 columns and was a wash.
- **Chain-of-thought distillation (sweep 21, -0.054)**: we generated brief (~50 token) reasoning traces from Claude Haiku 4.5 for each train example via `generate_cot_traces.py` (300/301 examples passed schema-grounding validation: the rationale had to reference at least one gold table). The CoT-trained adapter (~0.61) was strictly worse than sweep 13 (0.66) alone and adding it to the ensemble regressed by 0.007. Hypotheses: (a) the model's gradient budget is now spent learning to write reasoning tokens rather than getting more accurate at JSON tokens; (b) Haiku's short rationales are descriptive ("X contains Y") rather than structurally disambiguating, so they don't add new knowledge; (c) the model commits to a reasoning path early and is then constrained to match it – when the reasoning is wrong, the JSON is wrong too.

Which knobs mattered for table vs column F1. The ensemble's table score (0.7648) is 13 points above column score (0.6356) and this asymmetry is consistent across all our adapters. Sweep 13's r=32+MLP upgrade lifted column F1 the most among training changes (+0.045 vs sweep 12); embedding retrieval lifted column recall (+0.149 over BM25 in the 3-way ensemble). Going the other way, the SmolLM2 swap collapsed both axes evenly – suggesting column performance is bottlenecked by base-model capacity and retrieval quality, not by prompt or post-processing.

Surprises. The single most surprising result was that `types.keys` prompts hurt. We expected richer schema metadata to help; instead the model became less discriminative. The runner-up surprise was that `max_tables=40` hurt (-0.02) – BM25 retrieval was reasonable and exposing more distractors just confused the model. Together these argue that on ~1.5B with a small training set, *less context is more*.

What we would try next. (1) Retrain on embed-filtered prompts to close the small train/test retrieval-distribution gap (we infer with embed but the adapter was trained with BM25 prompts; expected +0.005–0.02). (2) Longer CoT traces (~200 tokens) with explicit table-disambiguation language – the brief CoT we tried in sweep 21 was too generic; a longer trace that names sibling tables and explains why each was/wasn't selected might actually teach the model new structure. (3) Hybrid retrieval with learned fusion weights instead of RRF, possibly with a small cross-encoder reranker over the union of BM25 + embed top-30 candidates.

5 AI Tools Statement

Code was developed iteratively with Anthropic's **Claude Code** (CLI agent on the **Claude Opus 4.7** model). Specific contributions: drafting `train_rapidfire.py`'s sweep configurations, investigating the rapidfireai SHM-checkpoint disk-flush bug (incl. reading `rapidfireai/fit/backend/worker.py` to identify the `num_chunks=1` workaround), implementing the embedding retriever, the LoRA-swap ensemble path in `main.py`, the post-processing utilities (`expand_columns.py`, `format_two_stage_data.py`, `reproduce_champion.py`), the SBO augmentation pipeline (`augment_data.py`) and the CoT-trace generator (`generate_cot_traces.py`) – both of which call the Anthropic API using **Claude Haiku 4.5** – and drafting this report. All design decisions, sweep selection, eval interpretation, and the final configuration choices were ours; the AI tool served as an implementation and analysis accelerant. The audit log `data/train_augmented_audit.json` records every generated example and its validation outcome for transparency.